

W3 PRACTICE

Express Basics + POST + Middleware

 *At the end of this practice, you can*

- ✓ **Create** and run a express.js HTTP server
- ✓ **Implement** route handling using express.js
- ✓ Parse form data from POST requests with middleware.
- ✓ Apply middleware concept to logging

 *Get ready before this practice!*

- ✓ **Read** the following documents to understand the nature of Express.js:
<https://expressjs.com/>
- ✓ **Read** the following documents to know more about Express.js's built-in middleware's:
<https://expressjs.com/en/resources/middleware.html>
- ✓ **Read** the following documents to understand MDN: HTTP POST:
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods/POST>
- ✓ **Read** the following documents to array filter:
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/filter

 *How to submit this practice?*

- ✓ Once finished, push your **code to GITHUB**
- ✓ Join the **URL of your GITHUB** repository on LMS



Ex01:— Refactoring

Goals

- ✓ Take advantage of Express.js framework's flexibility and minimalism
- ✓ Refactor code from node.js's built-in HTTP Module



Refactor the source code of EXERCISE 2 & 3 in Week 2 to Express.js

Q1 – What challenges did you face when using the native http module that Express.js helped you solve?

Q2 – How does Express simplify route handling compared to the native HTTP server?

Q3 – What does middleware mean in Express, and how would you replicate similar behavior using the native module?

Ex02: *The Express Foundation*

Goal: Initialize an Express application and start the server.

```
// 1. Import the express module
const express = _____('express');

// 2. Initialize the app
const app = _____();

app.get('/', (req, res) => {
  res.send('The foundation is solid!');
});

// 3. Start the server on port 3000
app._____(3000, () => {
  console.log('Server is running!');
});
```

Ex03: *The "ID" Hunter* (*req.params*)

Goal: Practice capturing dynamic segments of a URL path.

```
const express = require('express');
const app = express();

// Task: Make this route accept any student ID in the URL
// Example: /student/123 or /student/abc
app.get('/student/:_____', (req, res) => {

  // 1. Extract the ID from the request parameters
  const studentId = req._____.studentId;

  res.send(`Searching for student with ID: ${studentId}`);
});

app.listen(3000);
```

Ex04: *The Filter Specialist* (*req.query*)

Goal: Use query strings (the part after the ?) to "filter" a response.

```
const express = require('express');
const app = express();

// Test URL: http://localhost:3000/shop?item=laptop&price=1000
app.get('/shop', (req, res) => {

  // 1. Extract 'item' and 'price' from the query string
  const itemName = req._____.item;
  const itemPrice = req._____.price;

  res.json({
    product: itemName,
    cost: itemPrice
  });
});

app.listen(3000);
```

Ex05: *The "Next" Step* (*Middleware*)

Goal: Understand how middleware processes a request before it reaches the final route.

```
const express = require('express');
const app = express();

const simpleGuard = (req, res, _____) => {
  console.log("Someone is trying to access a route...");

  // 1. What function must you call to let the request continue?
  _____();
};

// 2. Apply the middleware globally
app._____(simpleGuard);

app.get('/protected', (req, res) => {
  res.send('You passed the guard!');
});

app.listen(3000);
```

Ex06: *Preparing for POST Data*

Goal: Configure Express to understand incoming JSON data (Crucial for Exercise 3).

```
const express = require('express');
const app = express();

// 1. Use the built-in middleware to parse JSON bodies
app.use(express.json());

app.post('/echo', (req, res) => {
  // 2. Access the data sent in the request body
  const receivedData = req.body;

  res.json({
    message: "I received your data!",
    data: receivedData
  });
});

app.listen(3000);
```

- Run your code from **Server**.
- In Apidog, create a **New Request** set to **POST**.
- URL: `http://localhost:3000/echo`.
- Go to the **Body** tab, select **JSON**, and type: `{"name": "Dara"}`.
- Hit **Send** and see if the server echoes it back!

Ex07: *The Logic Gate (Status Codes)*

Goal: Practice sending different HTTP status codes based on user input. This is essential for the "Invalid credit ranges" task in Exercise 2.

```
const express = require('express');
const app = express();

app.get('/check-score', (req, res) => {
  const score = req.query.score;

  if (score < 50) {
    // 1. Set the status code to 400 (Bad Request) and send a JSON error
    res.__(400).__({ error: "Score too low" });
  } else {
    // 2. Send a successful 200 status with a message
    res.__(200).send("You passed!");
  }
});

app.listen(3000);
```

Ex08: *The "Auto-File" Server (express.static)*

Goal: Understand how Express simplifies serving CSS, Images, or HTML files without manual routing.

```
const express = require('express');
const app = express();

/**
 * Task: Imagine you have a folder named "public" containing "index.html".
 * Instead of writing app.get('/index.html', ...), Express can serve the
 * whole folder automatically.
 */

// 1. Use the built-in middleware to serve the "public" folder
app.__(express.__( 'public' ));

app.listen(3000, () => {
  console.log("Try visiting http://localhost:3000/index.html");
});
```

- Enter `http://localhost:3000/check-score?score=40`.
- After hitting **Send**, look at the top right of the response panel.
- You will see **400 Bad Request** in red. This confirms your server is communicating the "state" of the request correctly to the client!

Ex09: API for Course Records

✦ For this exercise you will start with a **START CODE (EX-2)**

Goals

- ✓ Understand Route Parameters (:param)
- ✓ Work with Query Parameters (?key=value)
- ✓ Implement Conditional Logic for Filtering
- ✓ Build Real-World Web API Behavior
- ✓ Practice Defensive Programming

Context

You are building a backend API for a university's course catalog. Each course has the following fields

```
{
  "id": "CSE101",
  "title": "Introduction to Computer Science",
  "department": "CS",
  "level": "undergraduate",
  "credits": 3,
  "instructor": "Dr. KimAng",
  "semester": "fall"
}
```

Q1 - Create a route

```
GET /departments/:dept/courses
```

EXAMPLE

```
/departments/CSE/courses
```

Q2 - Accept query parameters to filter the result:

- `level` → e.g., undergraduate, graduate
- `minCredits` → integer
- `maxCredits` → integer
- `semester` → fall, spring, etc.
- `instructor` → partial match

EXAMPLE

```
/departments/CSE/courses?level=undergraduate&minCredits=2&semester=fall
```

Q3 - Return a JSON array of courses that match:

- The `:dept` from the route parameter
- The filter criteria from query parameters

Q4 – Handle Edge Cases

- **Invalid credit ranges** (minCredits > maxCredits)
- **No matching courses**
- **Missing or unsupported** query parameters (ignore them silently)

EXAMPLES

REQUEST
/departments/CSE/courses?level=undergraduate&minCredits=3&instructor=KimAng
RESPONSE
<pre>{ "results": [{ "id": "CSE101", "title": "Introduction to Data Science", "department": "CSE", "level": "undergraduate", "credits": 3, "instructor": "Dr. KimAng", "semester": "fall" }], "meta": { "total": 1 } }</pre>

Test cases

- `http://localhost:3000/departments/CSE/courses`
- `http://localhost:3000/departments/CSE/courses?level=undergraduate`
- `http://localhost:3000/departments/CSE/courses?minCredits=4`
- `http://localhost:3000/departments/CSE/courses?instructor=smith&semester=fall`

Ex10: *Enhance an API with Middleware*

Goal

Your goal is to modularize and secure your course filtering API using **Express middleware**. Middleware helps keep your code clean, reusable, and extensible.

Q1 - Create a middleware function that logs the following for every request:

- HTTP method (GET, POST, etc.)
- Request path (e.g., /departments/CSE/courses)
- Query parameters
- Timestamp in ISO format

✓ **Apply this middleware globally** so it logs **all incoming requests** to the server.

Q2 - Create a route-specific middleware to **validate query parameters**:

- If minCredits or maxCredits are present, ensure they are valid integers.
- If minCredits > maxCredits, return 400 Bad Request with an error message.

✓ **Apply this middleware only** to the /departments/:dept/courses route.

Q3 – (*Bonus*) Token-Based Authentication Middleware

Simulate basic API security:

- Require a token query parameter (e.g., ?token=xyz123)
 - If the token is missing or incorrect, respond with 401 Unauthorized.
- ✓ This middleware can be applied **either globally or to specific routes**.

Deliverables

- `logger.js` – contains your logging middleware.
- `validateQuery.js` – contains your validation middleware.
- `auth.js` (optional) – contains your token authentication middleware.
- `server.js` – where you apply middleware and define the course filtering route.

Test cases

```
GET /departments/CSE/courses?minCredits=abc
```

→ should return 400 Bad Request

```
GET /departments/CSE/courses?minCredits=4&maxCredits=2
```

→ should return 400 Bad Request

```
GET /departments/CSE/courses?token=xyz123
```

→ should succeed if token middleware is active

REFLECTIVE QUESTIONS

✦ For this part, submit it in separate PDF files

Middleware & Architecture

1. What are the advantages of using middleware in an Express application?
2. How does separating middleware into dedicated files improve the maintainability of your code?
3. If you had to scale this API to support user roles (e.g., admin vs student), how would you modify the middleware structure?

Query Handling & Filtering

4. How would you handle cases where multiple query parameters conflict or are ambiguous (e.g., `minCredits=4` and `maxCredits=3`)?
5. What would be a good strategy to make the course filtering more user-friendly (e.g., handling typos in query parameters like “falll” or “dr. smtih”)?

Security & Validation

6. What are the limitations of using a query parameter for authentication (e.g., `?token=xyz123`)? What alternatives would be more secure?
7. Why is it important to validate and sanitize query inputs before using them in your backend logic?

Abstraction & Reusability

8. Can any of the middleware you wrote be reused in other projects? If so, how would you package and document it?
9. How could you design your route and middleware system to support future filters (e.g., course format, time slot)?

Bonus – Real-World Thinking

10. How would this API behave under high traffic? What improvements would you need to make for production readiness (e.g., rate limiting, caching)?